

Копиращ конструктор:

Конструктор, който приема обект от същия тип и текущия става негово копие

- Ако не го разпишем, копирателен е дефолтен.

A obj; конструктор

A obj(); Ф-я

A copy(obj) // копи к-р

f(A obj)(i) -> копи к. на A

(j) -> ~A()

Оператор =:

- приема обект от същия клас и текущия става негово копие
- текущ обект е 3-и през копирател
- По 2, за да chain-не
- изтрива данните на обекта вляво



A obj;

A obj2 = obj; // копи к-р

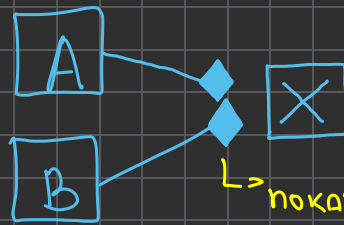
Композиция и Агрегация

I. Композиция (притежание)

```
class X {
    A obj1;
    B obj2;
};
```

Низнениет цикъл на obj1 и obj2 се контролира от X.

↓ UML Diagram



X трябва да укаже кои к-ри да се извикат на A и B.

↳ показва, че A и B са част от класа.

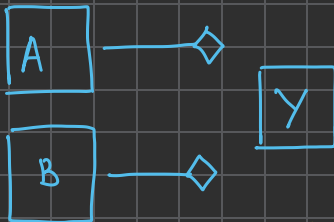
2. Агрегация - обект, който нямае независимо от нас

X не отговаря за низнениет цикъл на A.



```
class Y {
```

```
    A*
    B*
};
```



A и B могат да живеят извън рамките на Y.

↳ B кон-ра на X трябва да даде см-та на тази проф-я.

⚠️ Дескрипторът на X няма да извика дес-те на A и B, както и к-те.

Пример:

```
Config C;
App1.a1(c);
a1.run();
App2.a2(c);
a2.run();
```

Конструктор Y:

Y(A* ptr, B* ptr2):

{ a(ptr), b(*ptr2) }

~Y() не вика ~A() и ~B()

```
class X {
    A* ptr;
};
```

```
class Y {
    A & ref;
};
```

```
X(A* tmp): ptr(tmp)
~X() delete ptr;
```

```
Y(A & obj): ref(obj)
Y(A obj): ref(obj)
~A()
```

показваме по конте, на ref показваме по конте и в дес-ра се унищожават A obj

Абстракция:

- 1) ползваме нещо без да се интересуваме как работи
- 2) скриваме неужните детайли

Кансулация:

- ограничаване на достъпа

Модификатори за достъп:

- private
- protected (клас и наследници)
- public

→ class no default e
private
struct - no default e
public